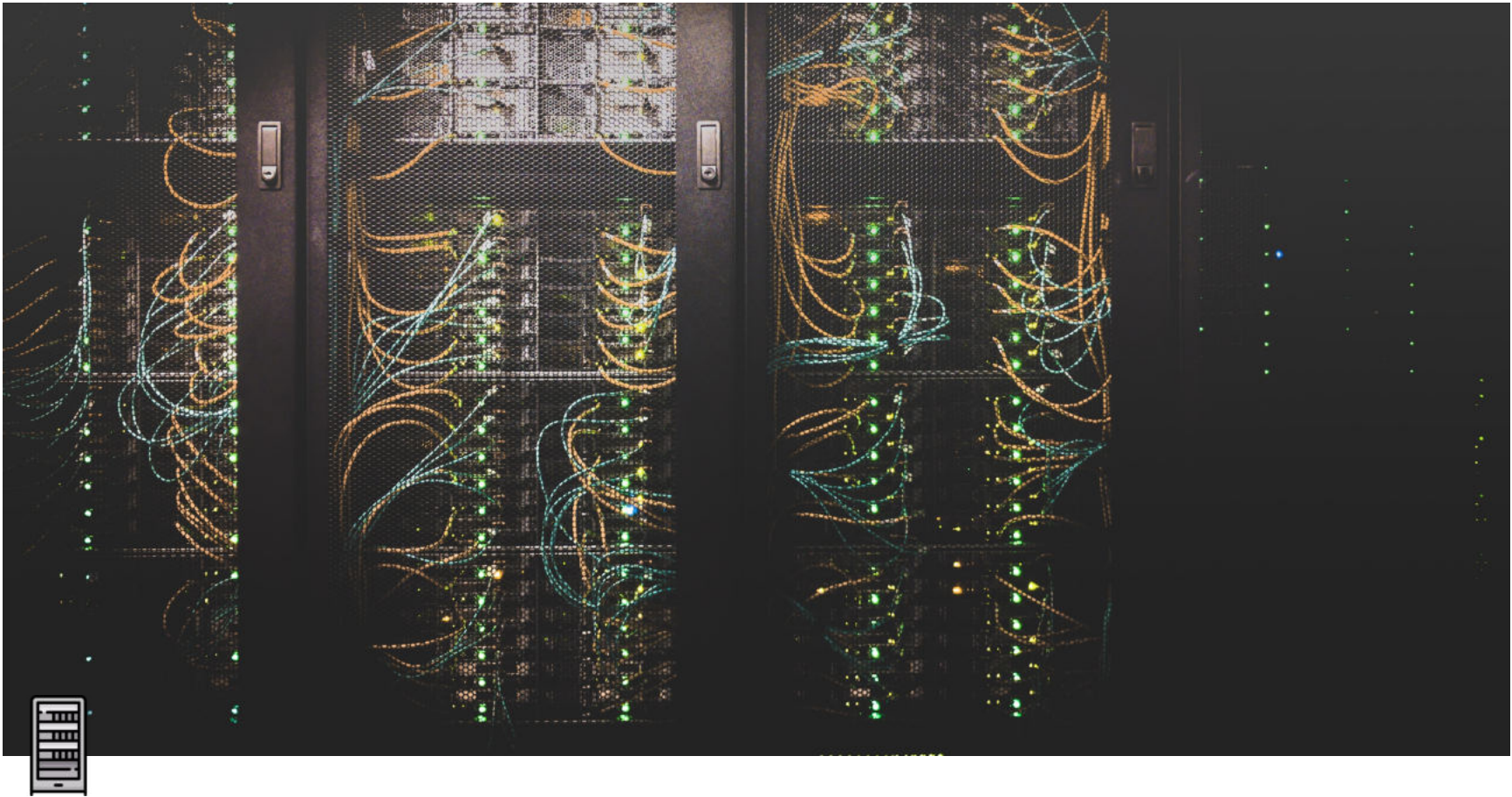




Linux Process Management

Type	Lecture
Date	@December 22, 2021
Lecture #	3
Lecture URL	https://youtu.be/2aThmDRvSWU
Notion URL	https://21f1003586.notion.site/Linux-Process-Management-d73af14458004045b281f5995be7cf32
Week #	2

sleep command

- It is a delay for a specified amount of time
 - The time is specified in seconds

Example

```
# The following command will make the prompt sleep for 5 seconds
sleep 5
```

Coprocess

- The shell command → coproc
- A Coprocess is executed asynchronously in a subshell
 - In simple words, this command is used to run a process without actually losing control of the prompt
 - As we don't lose prompt access, we can execute other commands

Usage

```
coproc sleep 20
```

When the above mentioned process is completed, the prompt will output a message saying Done

We can run the ps command to view the running status of the aforementioned command

kill

- This command is used to kill an already running process

- `ctrl + c` is also quite handy

Usage

```
kill -9 <PID>
```

How do I know the PID?

`ps` command, try `ps --forest` instead

& sign to put a process in the background

- We can also put the `&` (ampersand) sign at the end of a command to put that process in the background

Usage

```
sleep 30 &
```

Alright, how do I bring it to the foreground?

```
fg
```

jobs

List the commands that are running in the background

Example

```
kashif@Zen:~$ coproc sleep 30
[1] 4264
kashif@Zen:~$ coproc sleep 45
bash: warning: execute_coproc: coproc [4264:COPROC] still exists
[2] 4265
kashif@Zen:~$ jobs
[1]-  Running                  coproc COPROC sleep 30 &
[2]+  Running                  coproc COPROC sleep 45 &
```

top command

It's like your Windows Task Manager, but in the Linux terminal

To exit out of `top`, press `q` or `ctrl + c`

Pressing `ctrl + z` while `top` is open suspends the process, then you can do your work and come back to it by using the `fg` command

`jobs` command will show the top process

Well, `ctrl + z` will suspend any running process

\$-

```
echo $-
```

The output of this shell variable would tell us about the bash shell we are currently using

To know what the output means, type `man bash` and match the flags

Launch a bash shell which is not interactive

```
bash -c "echo \${-}"
```

```
bash -c "echo \${-}; ps --forest;"
```

To know the PID of the new bash shell we launcher

```
bash -c "echo \$$; echo \$-; ps --forest;"
```

history

The following command displays out all the commands that have been run on the current shell, chronologically

```
history
```

The following command will run the `n`-th command that has been run, make sure to put `n` from the list of commands output from the above command

`n` is integer

```
!n
```

The following command will run the previous command that was executed on the bash shell

```
!!
```

Brace Expansion

So the output of `echo $-` had a `B` flag, it refers to Brace Expansion

is this a JJK reference? ブレース展開

So, What is a Brace Expansion?

- Brace expansion is a mechanism by which arbitrary strings may be generated. [Source](#)

Example

```
echo a{b,c,d}e
```

```
kashif@Zen:~$ echo a{b,c,d}e
abe ace ade
```

```
echo {a..z}
```

```
kashif@Zen:~$ echo {a..z}
a b c d e f g h i j k l m n o p q r s t u v w x y z
```

```
echo {A..C}{g..k}
```

```
kashif@Zen:~$ echo {A..C}{g..k}
Ag Ah Ai Aj Ak Bg Bh Bi Bj Bk Cg Ch Ci Cj Ck
```

```
# The * is a wildcard character, it expands to all the files in the current dir
echo *
```

Similarly ...

```
# The following command will return all the files folders that start with D
echo D*
```

Exit codes

- `0` → All good, no error

- `1` → General errors, misc. errors
- `2` → Misuse of shell built-in
- `126` → Command invoked cannot execute
- `127` → Command not found
- `128` → Invalid argument to exit
 - `exit` takes int from `0 - 255`
- `128+n` → Fatal error signal "n"
 - When a process running in another place (like in another shell) was killed from somewhere else (killed from not the same shell)
 - `kill -9 $PPID` → `$?` returns `137` (`127 + 9`)
- `130` → Script terminated by `Ctrl + C`
- `255*` → Exit status out of range

If the exit code is out of range, bash takes the modulo of the exit code w.r.t. 256

Example

```
# Create a new bash subshell and exit with the status code 300
bash -c "exit 300"

# Check the exit code
echo $?

# The output will be ...
44
```